

AI_BRIDGE V2 — PROMPT DI CONTINUITÀ

Modalità di lavoro

CAVEMAN MODE FULL

- Risposte brevi.
 - Nessuna teoria inutile.
 - Nessuna proposta non richiesta.
 - Ogni risposta deve produrre un avanzamento concreto.
 - Se serve modificare un file:
 - fornire **file completo**;
 - mai patch;
 - mai frammenti;
 - includere compilazione;
 - includere test;
 - includere commit Git.
-

Repository

~/AI_Trading

Branch corrente:

feature/fast-track

Git locale.

Nessun remoto GitHub ancora configurato.

Regole fondamentali

Il repository è la fonte di verità.

La memoria della chat non prevale mai sul codice.

Prima di modificare:

1. verificare solo i file coinvolti;
2. evitare analisi dell'intero progetto;
3. nessun refactoring fuori milestone;
4. nessuna API inventata;

5. nessuna dipendenza inventata.

Stato progetto

Milestone completate:

- M01 Bootstrap
- M02 Runtime Controller
- M03 Application Layer
- M04 System Builder
- M05 Orchestrator
- M06 Market Engine
- M07 Decision Engine
- M08 Risk Engine
- M09 Memory Engine
- M10 Learning Engine
- M11 Diagnostics CLI
- M12 System Inspector
- M13 Continuity Dossier
- M14 Handoff Prompt
- M15 Project Progress

Milestone corrente:

M16 — System Diagnostics

Stato reale verificato

Esistono:

```
core_v2/application.py
core_v2/system_inspector.py
diagnostics_v2.py
```

Non esiste:

```
core_v2/system_diagnostics.py
```

`diagnostics_v2.py` usa esclusivamente funzioni pubbliche di `core_v2.application`.

`SystemInspector` espone già API di sola lettura sullo stato del sistema.

Decisioni progettuali confermate

Dalle verifiche del repository e delle chat precedenti risulta che:

- non è mai stata definita un'API pubblica per `SystemDiagnostics` ;
- non è mai stato scritto il file;
- non esiste pseudocodice;
- non esiste implementazione nascosta.

Quindi la nuova chat dovrà progettare **l'API minima**, coerente con l'architettura esistente, senza inventare dipendenze.

Obiettivo immediato

Completare M16.

Realizzare:

```
core_v2/system_diagnostics.py
```

Requisiti:

- usare `SystemInspector` ;
- nessuna logica di presentazione;
- raccogliere dati diagnostici;
- essere riutilizzabile da:
- `diagnostics_v2.py` ;
- Dashboard future;
- API future.

Successivamente integrare il modulo in:

```
core_v2/application.py
```

Lasciando la CLI dipendente solo dalle API pubbliche dell'application layer.

Metodo operativo

Ogni risposta deve terminare con:

- codice completo;
- compilazione;
- test;
- commit Git.

Mai interrompere il flusso con discussioni lunghe.

Produrre almeno un avanzamento verificabile per sessione.

Primo passo nella nuova chat

Leggere esclusivamente i file necessari alla M16.

Implementare direttamente `core_v2/system_diagnostics.py`.

Aggiornare `core_v2/application.py`.

Compilare.

Testare.

Commit.

Passare immediatamente alla milestone successiva.